Sensor Anomaly Detection with PCA

Overview

Learn normal sensor relationships and detect deviations using reconstruction error with a validation-set threshold.

Sensor Anomaly Detection with PCA

Use case and experiment

Detect unusual relationships among correlated sensors when labeled failures are scarce. Generate normal readings from two latent factors mapped to five sensors plus noise. Train on 700 normal rows and select a threshold from 250 separate normal rows. Test on 400 rows with 60 injected spikes in sensor four. This demonstrates novelty detection on a controlled stationary process.

Sensor Anomaly Detection with PCA

Model mathematics

Standardize sensors using training mean and standard deviation. Singular value decomposition identifies orthogonal directions of variation. Keep the first two principal components. Project a standardized observation into the two-dimensional subspace and reconstruct it in five dimensions. Anomaly score is the mean squared difference between the original standardized row and its reconstruction. Large residuals indicate unusual cross-

sensor relationships.

Sensor Anomaly Detection with PCA

Evaluation and leakage prevention

Training contains no injected anomalies. Threshold is the 99th percentile of normal validation reconstruction errors, without reading test labels. Flag scores strictly above that threshold. Report test precision, recall, F1 and confusion counts, and compare with an always-normal baseline. The intended validation tail fraction is not a guaranteed false-positive rate on future data. With only 250 validation points, extreme quantile

estimates are uncertain.

Sensor Anomaly Detection with PCA

Files, outputs and inference

project.py exposes normal(), fit(), score() and main(). results/model.npz stores training mean, scale, retained components and threshold. To score new rows, preserve the five-sensor order and original measurement units, load the model and call score(). Compare errors to the saved threshold. results/test_scores.csv contains test readings, injected labels and reconstruction errors. metrics.json and report.html summarize performance.

Sensor Anomaly Detection with PCA

Interview questions and extensions

How is PCA different from a classifier? It learns a subspace rather than a direct class boundary. Why normalize? Large-scale sensors otherwise dominate. Can it miss anomalies? Yes: unusual points lying inside the learned subspace may reconstruct well. Why not fit on the full dataset? Test distribution and anomalies would influence preprocessing and components. Extend with real licensed sensor data, chronological evaluation, drift

monitoring, residuals per sensor, rolling thresholds and comparisons with robust distance or isolation methods. Validate alarms with domain experts.

Sensor Anomaly Detection with PCA

Measured results from this build

data: "700 normal training, 250 normal validation, 400 test rows including 60 injected anomalies; five synthetic sensors"

threshold: 0.019620611832701626

test: {"accuracy": 0.9975, "precision": 0.9836065573770492, "recall": 1.0, "f1": 0.9917355371900827, "confusion_matrix": {"tn": 339, "fp": 1, "fn": 0, "tp": 60}}

always_normal_baseline: {"accuracy": 0.85, "precision": 0, "recall": 0.0, "f1": 0, "confusion_matrix": {"tn": 340, "fp": 0, "fn": 60, "tp": 0}}

score_summary: {"normal_mean": 0.004468385141704088, "anomaly_mean": 1.5605930392274352}

caveat: "Detects deviations outside the learned subspace. In-subspace anomalies and drift can be missed; injected spikes are deliberately simple."

See results/metrics.json for the complete machine-readable output, including individual predictions where available.

Sensor Anomaly Detection with PCA

Setup and repository map

Requires Python 3.10 or later and NumPy. Create a virtual environment with python -m venv .venv. On Windows activate it with .venv\Scripts\Activate.ps1, or use .venv\Scripts\python.exe directly if activation is restricted. Install dependencies using python -m pip install -r requirements.txt. Run python project.py to train and evaluate, then open results/report.html in a browser. Run python -m unittest discover -v for tests. common.py

handles shared metrics and reporting; project.py contains the model and experiment; test_project.py contains four checks. Results are generated locally. No paid API, GPU or external dataset download is required.

Reproducibility, GitHub and honest presentation

Random seeds make the experiment reproducible within normal numerical-library differences, but exact floating-point results can vary by NumPy build. Runtime requires numpy>=1.26,<3. The shipped metrics record the environment used for this build in BUILD_INFO.txt. A GitHub Actions workflow runs tests and the training experiment on Python 3.11. Keep API keys and private datasets out of Git. The MIT license permits reuse. This is an AI-

assisted educational starter: explain what you reviewed, modified and tested. Synthetic labels are deliberately generated from model-friendly processes, so favorable scores are not evidence of real-world generalization. Learn one project deeply and add a documented extension before presenting it as your work.